AI reads a class histogram

Paste two histograms and ask what moved, not what a histogram is.

```
Here are two `jcmd GC.class_histogram -all` outputs, before and after one request.
Which classes grew most, in instances and in bytes?
Which are boxes, map nodes, arrays or lambdas, and what code likely made them?
```

- A histogram holds class names and counts; a heap dump holds your data, keep it local.
- Ask for the arithmetic, then check two rows yourself.

AI reads JOL and JMH results

Raw JMH and JOL output is long; an assistant turns it into a comparison.

- Give it the raw files (`-rf text`, JOL printouts), not a summary.
- Ask for bytes per call by variant: headers off and on, escape analysis off.
- Ask which numbers are measured and which are derived.
- Every number on these slides traces back to a raw file under `generated/runs`.

A skill to repeat the check


```
---
name: heap-check
description: Compare class histograms of a running JVM before and after a workload
---
1. Run `jps -l` and pick the pid of the app.
2. Run `jcmd <pid> GC.class_histogram -all` before and after the workload.
3. Subtract per class; list the top 10 by bytes.
4. Flag boxes, `HashMap$Node`, `Object[]` and `$$Lambda` classes.
5. Compare with the last baseline in `heap-baseline.txt`.
```

- Run it every release, not only when the basement floods.

DEMO Demo: who moved my heap?

The histogram from the first demo, pasted into an AI assistant.

```
This is `jcmd <pid> GC.class_histogram` of a running JVM.
Who moved my heap? Name the top 5 classes by bytes, say which are
boxes, nodes, arrays or lambdas, and what I could change.
```